


ENSEA

Beyond Engineering

# REAL-TIME OPERATING SYSTEMS

## Session 2 – Queues

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

# Where we are

- Session 1: `bsp_process` and `app_process` in two tasks
  - `task_bsp`: priority 2, polls inputs every 5 ms, **blocks** the rest of the time
  - `task_app`: priority 1, **busy loop** – still polls our home-made event queue
- Today:
  - Our demo has a **bug**. Let's find it
  - Replace the home-made queue by a **FreeRTOS queue**
  - Make the APP **wait** for its events: the **Active Object** pattern

ENSEA
Beyond Engineering

# Today

1 A Bug in Our Demo

2 FreeRTOS Queues

3 Queues and Interrupts

4 The Active Object Pattern

ENSEA
Beyond Engineering

# Our event queue, again

**Producer:** `task_bsp`

```
bool event_queue_push(event_queue_t *q,
                      app_event_t evt) {
    if (q->count >= EVENT_QUEUE_CAPACITY)
        return false;
    int tail = (q->head + q->count)
             % EVENT_QUEUE_CAPACITY;
    q->items[tail] = evt;
    q->count++;
    return true;
}
```

**Consumer:** `task_app`

```
bool event_queue_pull(event_queue_t *q,
                      app_event_t *out) {
    if (q->count == 0)
        return false;
    *out = q->items[q->head];
    q->head = (q->head + 1)
            % EVENT_QUEUE_CAPACITY;
    q->count--;
    return true;
}
```

## Question

Two tasks, one shared variable `q->count`, and a scheduler that can switch tasks *anywhere*. What can go wrong?

ENSEA
Beyond Engineering

# count++ is not one instruction

q->count++ compiles to:

```
LDR r3, [count]
ADDS r3, r3, #1
STR r3, [count]
```

Read – modify – write. A context switch can happen between any two instructions.

| task_app **(pull)** | task_bsp **(push)** | count |
|---|---|---|
| LDR r3, [count] ($r3 = 2$) | | 2 |
| *– preempted by the BSP –* | | |
| | LDR r3, [count] ($r3 = 2$) | 2 |
| | ADDS r3, r3, #1 ($r3 = 3$) | 2 |
| | STR r3, [count] | 3 |
| | *– blocks (vTaskDelay) –* | |
| SUBS r3, r3, #1 ($r3 = 1$) | | 3 |
| STR r3, [count] | | **1** |

- Expected: $2 + 1 - 1 = 2$. We get **1**: the BSP's event is **lost**
- Each task saved its own r3: the context switch works perfectly. The bug is in *our* code

ENSEA
Beyond Engineering

# A race condition

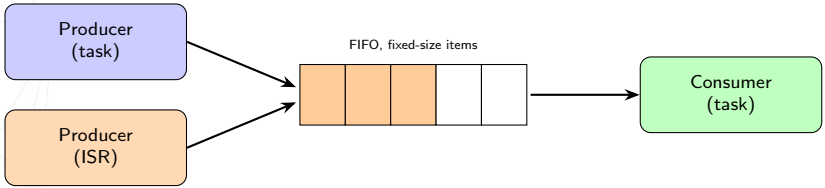
- In the superloop, push and pull could never overlap: `bsp_process` and `app_process` ran *one after the other*
- With preemption, the BSP can interrupt the APP *in the middle* of `count--`
- The window is a few instructions wide: the bug is **rare**
  - Invisible in a demo, impossible to reproduce on demand
  - But real: one day, a click is lost (or worse, the queue is corrupted)

## Race condition

The result depends on the *timing* between tasks. Session 3 names and fixes this problem in general (reentrancy). Today, we use a queue that is already protected.

ENSEA
Beyond Engineering

# Same idea, done right




- Same principle as our event queue: a FIFO of fixed-size items
- Two big differences:
  - **Thread-safe**: several tasks (and ISRs) can use it at the same time without corrupting it
  - **Blocking**: a task can *wait* on it, in the Blocked state, without using the CPU

ENSEA
Beyond Engineering

# The API

```c
#include "queue.h"

/* Consumer's init */
QueueHandle_t q = xQueueCreate(16, sizeof(app_event_t));   /* 16 items, NULL if no heap */

/* Producer */
app_event_t evt = { .type = EVT_SW1_CLICK };
xQueueSend(q, &evt, timeout);           /* copy evt INTO the queue (at the back)  */

/* Consumer */
app_event_t received;
xQueueReceive(q, &received, timeout);   /* copy the oldest item OUT of the queue */
```

- The queue is created in the FreeRTOS heap: check that `xQueueCreate` did not return `NULL`
- Items are passed by **address**, but stored by **copy** (next slides)

ENSEA
Beyond Engineering

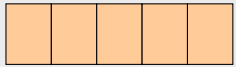
# When does a task block?

## Receive on an **empty** queue



The task is Blocked until an item arrives

## Send to a **full** queue



The task is Blocked until there is room

| **Timeout** | **Meaning** |
|---|---|
| 0 | do not wait: return immediately |
| `pdMS_TO_TICKS(10)` | wait at most 10 ms |
| `portMAX_DELAY` | wait forever |

- Return value: `pdPASS`, or `errQUEUE_FULL` (send) / `pdFALSE` (receive) after a timeout – **check it**

ENSEA
Beyond Engineering

# Copy by value

- `xQueueSend` copies `sizeof(item)` bytes into the queue
- `app_event_t` is small (a type + a data word): perfect, the sender can forget it right away

**Sending a pointer: only the pointer is copied**

```
void send_name(QueueHandle_t q) {
    char name[16] = "dino";
    char *p = name;
    xQueueSend(q, &p, 0);
}   /* name is gone: p points to garbage */
```

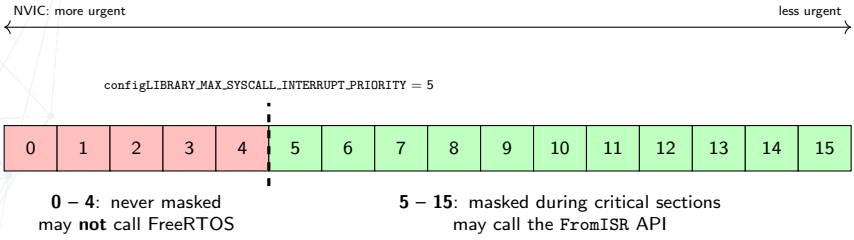
## Lifetime

If you send a pointer, the pointed data must still exist when the receiver reads it: `static`, global, or a buffer whose owner is clearly defined.

ENSEA
Beyond Engineering

# How is it thread-safe?

- Inside `xQueueSend` / `xQueueReceive`, FreeRTOS enters a **critical section**: no context switch, no interfering ISR
- But it does **not** mask *all* interrupts:




- The most urgent interrupts are **never delayed** by the kernel – the price: they cannot talk to it

ENSEA
Beyond Engineering

# Sending from an ISR

```
void HAL_GPIO_EXTI_Callback(uint16_t pin) {          /* runs in interrupt context */
    BaseType_t woken = pdFALSE;
    app_event_t evt = { .type = EVT_SW1_CLICK };

    xQueueSendFromISR(q, &evt, &woken);   /* never blocks: no timeout parameter */
    portYIELD_FROM_ISR(woken);            /* switch NOW if a more urgent task woke up */
}
```

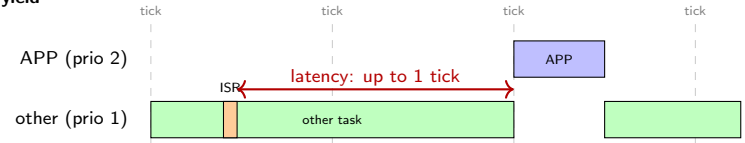
Peux échouer si il n'y a plus de place dans la queue

- An ISR **cannot block**: it is not a task, the scheduler cannot put it to sleep
- Every API usable in an ISR has a `FromISR` variant. **Never** call the normal one from an ISR
- `woken` is set to `pdTRUE` if the send unblocked a task more urgent than the interrupted one

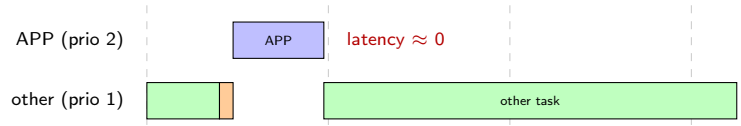
ENSEA
Beyond Engineering

# Why portYIELD_FROM_ISR?

**Without yield**




**With yield**




- Without the yield, the ISR returns to the interrupted task: the woken task waits for the next tick – back to time-slicing

ENSEA
Beyond Engineering

# Which interrupts may call FreeRTOS?

- Only those with a NVIC priority **numerically** $\geq$ **5** (`configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY`)
- Reminder: for the NVIC, **0 is the most urgent** – the opposite of task priorities
- In our project, CubeMX puts our IRQs at 5: allowed
- An interrupt that is too urgent and calls FreeRTOS:
  - caught by `configASSERT` if you are lucky
  - silent corruption of the kernel if you are not

**Rule**

Before calling a `FromISR` function, check the priority of the interrupt in CubeMX (NVIC tab).

ENSEA
Beyond Engineering

# The queue is full, in an ISR: now what?

- `xQueueSendFromISR` has no timeout: it returns `errQUEUE_FULL` and the event is **lost**
- Waiting in the ISR is **forbidden**: the consumer task cannot run during the ISR, room will never appear

## Key message

A full queue is a **design** problem, not an API problem.

1. **Size it so that it does not happen**
   - A full queue is a symptom: the consumer is too slow, or blocked for too long
   - Size $\approx$ producer rate $\times$ worst-case consumer latency, plus a margin
2. **Never lose silently**
   - Count overflows in the ISR, or `configASSERT` in debug
   - A visible loss gets fixed. A silent one becomes "the button does not always work"

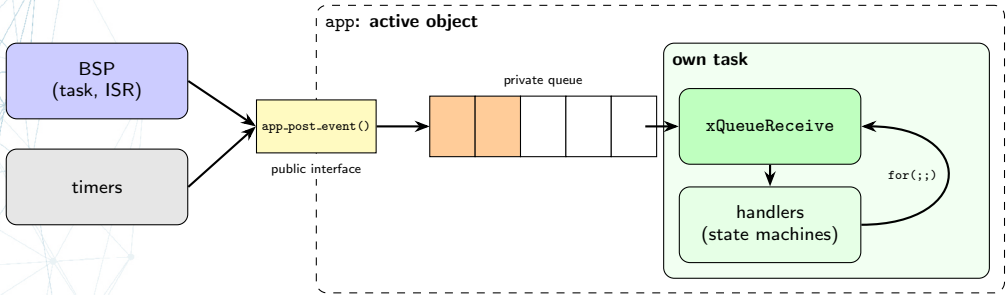
ENSEA
Beyond Engineering

# The queue is full, in an ISR: now what? (continued)

3. **Adapt the strategy to the nature of the event**
   - **Occurrence** (a click): every event counts
     - Keep a counter or a "click pending" flag in the ISR, post it later
   - **State** (encoder position): only the latest value counts
     - Aggregate: accumulate the delta, post only when no event is pending (our `bsp_process` already does it)
     - Or a mailbox: queue of length 1 + `xQueueOverwriteFromISR` – the last value wins
4. **Choose what to drop**
   - By default, the **newest** event is dropped (the send fails)
   - Dropping the oldest: `xQueueReceiveFromISR`, then send again – possible, rarely justified

ENSEA
Beyond Engineering

# Active Object




- An object that owns its **data**, its **task** and its **queue**
- Nobody touches its data from outside: others only **post events**
- Its task waits on the queue and handles events **one at a time**: no race condition inside the object
- We already had the events and the state machines: we add a task and a real queue

ENSEA
Beyond Engineering

# The object creates its own task

```
typedef struct {
    TaskHandle_t task;          /* kept in the object (needed for notifications) */
    QueueHandle_t events;       /* private queue of the active object */
    /* ... timers, mode, menus, runner ... as before */
} app_t;

static app_t app;               /* singleton, private to app.c */
```

+ Pratique que d'avoir une app global (variable global)

```
void app_init(unsigned int task_priority) {
    /* ... same init as before ... */
    app.events = xQueueCreate(16, sizeof(app_event_t));
    configASSERT(app.events != NULL);

    BaseType_t ok = xTaskCreate(app_task, "APP", 256, NULL, task_priority, &app.task);
    configASSERT(ok == pdPASS);
}
```

- The queue and the task are created in `app_init`: `main` no longer knows that the APP has a task
- app is a singleton: the task does not need `pvParameters` to find its object (`NULL`)
  - With several instances, `pvParameters` would carry a pointer to the object

ENSEA
Beyond Engineering

# The task loop

```
static void app_task(void *unused) {
    (void)unused;
    app_event_t evt;

    for (;;) {
        if (xQueueReceive(app.events, &evt, pdMS_TO_TICKS(10)) == pdPASS) {   /* Blocked here */
            app_handle_event(&evt);                          /* the old body of app_process */
        }
        app_timers_process(&app.timers, bsp_get_tick_ms());   /* timers: still polled */
    }
}
```

- The only blocking point of the task: `xQueueReceive`
- Timeout of 10 ms: our timers are still polled, the task must wake up to check them
  - With `portMAX_DELAY`, no timer would ever expire
  - A later session replaces them with FreeRTOS software timers

ENSEA
Beyond Engineering

# Posting events

```c
/* app.h: the public interface of the active object */
bool app_post_event(app_event_t evt);            /* from a task */
bool app_post_event_from_isr(app_event_t evt);   /* from an ISR */

/* app.c */
bool app_post_event(app_event_t evt) {
    return xQueueSend(app.events, &evt, 0) == pdPASS;
}

bool app_post_event_from_isr(app_event_t evt) {
    BaseType_t woken = pdFALSE;
    BaseType_t ok = xQueueSendFromISR(app.events, &evt, &woken);
    portYIELD_FROM_ISR(woken);
    return ok == pdPASS;
}
```

- The BSP calls `app_post_event`: it no longer sees the queue, nor FreeRTOS
- The task / ISR distinction is visible in the API. Returns `false` if the queue is full
- Going from bare-metal to RTOS only changes `app.c` (and `main.c`)

ENSEA
Beyond Engineering

# Why timeout 0?

## The APP posts to its own queue

The timers are processed by `app_task`, and they post `EVT_TIMER` through `app_post_event`. If the APP blocked on its own full queue, it would wait for itself, forever.

- The BSP does not block either: a lost event is reported by the return value
- Remember: a full queue is a **design** problem

ENSEA
Beyond Engineering

# And the BSP? A module with its own task

- `bsp_init(2)` creates its own task too, like `app_init(1)`
  - `main` only chooses the **priorities** (a system decision: who runs first) and starts the scheduler
  - The stack size stays inside each module: it depends on the module's code
- **Not** an active object: no queue, no events
  - The BSP is **time-driven**: it wakes up every 5 ms
  - The APP is **event-driven**: it wakes up on a message
- The BSP now depends on FreeRTOS: fine, it is platform-specific by nature
  - The SDL BSP would implement the same `bsp.h` its own way
- The init order stays safe
  - `bsp_init` creates its task before `app_init` creates the queue...
  - ... but no task runs before `vTaskStartScheduler()`: no post on a `NULL` queue
- `vTaskDelay(5)` after the processing: period = 5 ms + execution time
  - Irrelevant for polling buttons

ENSEA
Beyond Engineering

# Design rule: run-to-completion

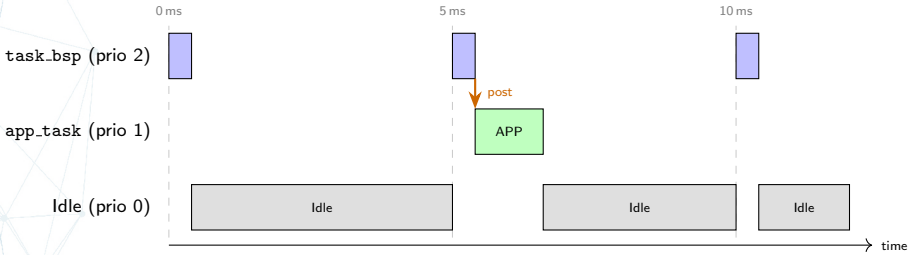
## Run-to-completion

An event handler runs until it is done, and **never blocks**.

- The only place where the task waits is `xQueueReceive`
- A handler that blocks (delay, waiting for a peripheral...) freezes the whole object
  - Events pile up, the queue fills, events get lost
- Same idea as our state machines: a transition is a **short** piece of work
- Long work? Split it into several events, or give it to another task

ENSEA
Beyond Engineering

# Result




- The APP runs only when there is something to do. Idle finally runs
- Both tasks block: priorities now **mean** something
- The CPU is free most of the time: room for more tasks, or for low-power modes

ENSEA
Beyond Engineering

# Summary

- Sharing a variable between preemptible tasks creates **race conditions**: rare, but real
- A FreeRTOS queue is **thread-safe** and **blocking**; items are **copied**
- Timeouts: `0`, a duration, or `portMAX_DELAY` – and always check the return value
- From an ISR: `FromISR` variants, `portYIELD_FROM_ISR`, and only interrupts with priority $\geq 5$
- A full queue is a design problem: size it, never lose silently, adapt to occurrence / state
- **Active Object**: data + task + private queue; others only post events; handlers **run to completion** = ne jamais bloquer
- Next: synchronisation – notifications, semaphores, mutexes

ENSEA
Beyond Engineering